Semana 2 · Programación del ESP32

Unidad 1 · variables, tipos, condiciones, ciclos, funciones y juego de luces

1. ¿Cómo se ejecuta un programa?

Programa -> instrucciones ordenadas. El compilador traduce el código a instrucciones que puede ejecutar el microcontrolador.

↳ Para que el ESP32 haga algo primero necesito indicarle los pasos. Si el código tiene errores al escribirlo, ni compila; si está bien escrito pero planteé mal la condición, puede funcionar diferente.

Escribir código -> compilar -> cargar al ESP32 -> ejecutar.

CPU -> busca, decodifica y ejecuta. El programa se repite o responde a eventos según su lógica.

- En un programa típico se configura primero el hardware y después se realizan las tareas repetitivas.
- Error de sintaxis -> no compila. Error lógico -> compila, pero hace algo distinto a lo esperado.

Nota al margen: Primero revisar el circuito; después el programa. A veces el problema está en un cable...

2. Variables, tipos y constantes

Variable -> espacio de memoria donde se guarda un valor que puede cambiar durante la ejecución.

↳ En una variable puedo guardar, por ejemplo, cuántas veces se presionó un botón. Ese valor cambia durante el programa; una constante la dejo fija, como el número de un pin.

int -> entero (0, 1, 25, -3). float -> número con decimales.

char -> carácter. bool -> true / false.

const -> valor fijo, no debería modificarse después.

```
int contador = 0;
float voltaje = 2.5;
bool encendido = false;
const int pinLed = 2;
```

- Asignar (=) -> guardar un valor. Comparar (==) -> preguntar si dos valores son iguales.
- Importante -> una variable local solo se usa dentro de su bloque o función; una global puede compartirse, con cuidado.

Operadores que se usan mucho

+ - * / -> cálculos. % -> residuo.

> < >= <= == != -> comparaciones.

&& -> Y lógico. || -> O lógico. ! -> negación.

contador++ -> aumenta 1. contador-- -> disminuye 1.

3. Decisiones: if, else y switch

Condicional -> ejecuta una acción solo si se cumple una condición.

↳ Lo uso para decidir: si el botón está presionado → enciendo; caso contrario → apago. Ojo con = y ==, porque uno asigna y el otro compara.

if -> si es verdadero. else -> de lo contrario. else if -> otra opción.

```
int lectura = 1;
if (lectura == 1) {
  // encender una salida
} else {
  // mantenerla apagada
}
```

switch -> útil cuando se compara un valor con varios casos concretos.

- Ejemplo: botón presionado -> LED encendido; botón libre -> LED apagado.

Nota al margen: Usar == para preguntar; con = se asigna...

4. Bucles / repeticiones

for -> se sabe cuántas vueltas se van a dar.

↳ Los ciclos sirven para evitar escribir la misma instrucción varias veces. Con for conozco cuántas repeticiones quiero; con while depende de una condición...

while -> repite mientras la condición se cumpla.

do while -> ejecuta al menos una vez y luego revisa la condición.

```
for (int i = 0; i < 3; i++) {
  // repetir tres veces
}

while (contador < 5) {
  contador++;
}
```

Ojo -> si una condición nunca cambia, el bucle puede quedar ejecutándose sin terminar y otras tareas no se atienden.

5. Funciones y organización del código

Función -> bloque que reúne instrucciones para no repetir el mismo código en varias partes.

↳ Separar el programa en funciones ayuda a entender qué hace cada parte. Una función podría controlar luces y otra revisar una entrada, así no mezclo todas las instrucciones.

Nombre + parámetros -> recibe datos. return -> devuelve un resultado, si corresponde.

void -> función que no devuelve un valor.

```
void encenderLed(int pin) {
  digitalWrite(pin, HIGH);
```

```
}

void apagarLed(int pin) {
  digitalWrite(pin, LOW);
}
```

setup() -> configura pines al iniciar. loop() -> repite la lógica principal.

pinMode(pin, OUTPUT) -> salida. pinMode(pin, INPUT_PULLUP) -> entrada con resistencia de elevación interna.

digitalWrite -> cambia salida. digitalRead -> lee entrada.

delay(ms) -> espera en milisegundos, pero detiene temporalmente el flujo normal.

6. Aplicación: juego de luces con ESP32

Idea de práctica del sílabo -> varios LED que cambian por turnos. Para comprender variables, arreglos, ciclos y tiempos.

ESP32 -> GPIO 18 / 19 / 21 -> resistencias -> LED.

Secuencia -> encender LED 1 -> apagar -> LED 2 -> apagar -> LED 3...

Código base de estudio (ejemplo, no resultado de montaje)

```
const int leds[] = {18, 19, 21};
const int total = 3;

void setup() {
  for (int i = 0; i < total; i++) {
    pinMode(leds[i], OUTPUT);
  }
}

void loop() {
  for (int i = 0; i < total; i++) {
    digitalWrite(leds[i], HIGH);
    delay(300);
    digitalWrite(leds[i], LOW);
  }
}
```

- Array leds[] -> guarda los pines. Variable i -> recorre posiciones 0, 1 y 2.
- Cambiar delay(300) -> cambia la velocidad de la secuencia. 300 ms = 0,3 s.
- Si se usa pulsador -> digitalRead permite escoger otra secuencia; revisar rebote y entrada flotante.

7. Errores que pueden salir

No enciende -> pin incorrecto / LED invertido / falta GND / resistencia / código.

Se prende distinto -> revisar el orden del arreglo y los cables.

Simulación en Wokwi -> ayuda a comprobar conexiones y lógica, pero revisar igual el montaje real.

No usar el mismo GPIO como entrada y salida a la vez sin volver a configurarlo.

Ejemplo rápido: leer un pulsador

Con INPUT_PULLUP la lectura normal es HIGH; al conectar el botón a GND pasa a LOW. Así se puede decidir qué hacer sin dejar el pin flotante.

```
const int boton = 18;
const int led = 2;

void setup() {
  pinMode(boton, INPUT_PULLUP);
  pinMode(led, OUTPUT);
}

void loop() {
  bool pulsado = (digitalRead(boton) == LOW);
  digitalWrite(led, pulsado ? HIGH : LOW);
}
```

Operador ?: -> una condición corta: si se pulsa, HIGH; si no, LOW.

Después se puede combinar botón + secuencia de tres luces para cambiar el patrón.

Resumen corto

Variable -> dato. if -> decisión. for -> repite. función -> organiza. GPIO -> controla.

Pendiente de clase -> anotar los pines reales y qué secuencia se usó...